TAR System Description Paper Template

Anonymous submission

University of Zagreb, Faculty of Electrical Engineering and Computing
Unska 3, 10000 Zagreb, Croatia
{author1,author2,author3}@fer.hr

Abstract

This document provides the instructions on formatting the TAR system description paper in LaTeX. This is where you write the abstract (i.e., summary) of the work you carried out within the project. The abstract is a paragraph of text ranging between 70 and 150 words.

1. Introduction

Text for the introduction goes here.

2. Related work

Last decade's work by Cao and Yang (2015) formalised the idea of machine unlearning, establishing a new research area that has quickly grown to include large language models (LLMs). Since then, many papers exploring this concept have been published, and several unlearning methods have been developed.

Notably, Zhang et al. (2024) introduced Negative Preference Optimisation (NPO) as a novel unlearning method. It is more stable and serves as an effective alternative to the existing Gradient Ascent method. To reduce the large computational overhead caused by full-parameter updates during optimisation, parameter-efficient unlearning techniques have become crucial. For example, Abitante et al. (2026) use low-rank adaptation (LoRA) to enhance the model's robustness to quantisation, while also significantly cutting down the number of trainable parameters.

In contrast to weight-modification methods, inference-time alignment offers an alternative approach. Turani et al. (2026) demonstrated that it is possible to suppress information without having to modify the model's internal parameters. They applied vector steering directly to the hidden states at inference time.

Alongside the development of unlearning methods, substantial effort has gone into creating standardized benchmarks for evaluation. Several studies have designed such benchmarks to compare different unlearning methodologies (Maini et al., 2024; Shi et al., 2025; Anna et al., 2026). Furthermore, Lynch et al. (2024) survey the possibilities and limitations of existing benchmarks, emphasizing the need to test the robustness of unlearning through adversarial techniques, such as jailbreak prompting.

While past research usually evaluates optimization, parameter efficiency, or inference-time steering separately, our work connects these different views. Unlike earlier methods, we create a unified framework that combines NPO with LoRA for computationally efficient unlearning, and benchmark this setup against a baseline model that has not underwent through the unlearning process. Additionally, we implement the Activation Steering method to test whether we can improve the unlearning effectiveness at inference-time. Furthermore, while existing studies often overlook adversarial weaknesses, we introduce a detailed seven-tier red-teaming protocol to systematically evaluate the limits of model unlearning robustness against jailbreak attacks under weight-modified configurations.

3. Methodology

In this section, we describe the established framework for evaluating the effectiveness and robustness of pairing NPO with LoRA for unlearning in large language models. We also define the evaluation framework for the adversarial jailbreak attacks. We detail the datasets, models, and evaluation metrics used to assess the performance of this approach.

3.1. Tasks and dataset

The primary dataset used in our experiments is part of the *LUME: LLM Unlearning with Multitask Evaluations* benchmark (Ramakrishna et al., 2025). The dataset includes documents that are categorised into three scenarios:

(I) **Synthetic creative documents** These documents are synthetic fictional stories in various genres.

(II) **PII-containing synthetic biographies** These are synthetic biographies of public figures which contain personally identifiable information (PII), including names, home addresses, and Social Security numbers.

(III) **Real documents from the pretraining corpus** These are real documents that were included in the pretraining corpus of the language model.

Each subtask has two main data splits: *Forget* and *Retain*. The *Forget* split consists of documents that the model should unlearn, while the *Retain* split includes documents that should be retained in the model's memory. The number of examples can be found in Table 1.

Table 1: Distribution of documents in the dataset.

Document	Forget	Retain	Total
Synthetic Creative Documents	166	206	372
Synthetic Biographies	642	612	1254
Real Documents	304	318	622
Total	1112	1136	2248

3.2. Model

We conduct our experiments using the *OLMo-7B-0724-Instruct-hf* model (Groeneveld et al., 2024), which has been explicitly fine-tuned to memorise the documents from the LUME benchmark dataset.

Unlearning the above-mentioned model is achieved through NPO. Since this method is computationally heavy, we use it in combination with LoRA to decrease the number of trainable parameters, making the unlearning process more efficient. All experiments were run on a single NVIDIA A100 40GB GPU.

Given a prompt x, the training objective of our approach is to minimise the likelihood of generating forgotten documents y_f, compared to a fixed reference model π_{ref}. Formally, the training objective can be expressed as follows:

$$\mathcal{L}_{NPO}(\theta) = \frac{2}{\beta} \log \left(1 + \left(\frac{\pi_\theta(y_f|x)}{\pi_{ref}(y_f|x)} \right)^{\beta} \right),$$

where π_θ is the model being trained, π_{ref} is the reference model, and β is a hyperparameter that controls the strength of the preference. Additionally, to maintain the model's general capabilities, we include an additional cross-entropy loss term that encourages the model to keep the knowledge of the retained documents y_r. The final loss function is:

$$\mathcal{L}(\theta) = \mathcal{L}_{NPO}(\theta) + \alpha \cdot \mathcal{L}_{CE}(\theta),$$

where $\mathcal{L}_{CE}(\theta)$ is the cross-entropy loss computed on the retained documents, and α is a hyperparameter that aids in balancing unlearning and knowledge retention.

As our inference time unlearning method, we implemented Activation Steering method, which was proposed in (Stolfo et al., 2024). Activation steering identifies a desired direction in the model's latent space and surpresses or enhances it at inference time. Given a set of N pairs (x_f, x_r), where x_f represents a forget-set prompt and x_r represents a retain-set prompt, we extract hidden-state activations h_f and hr at decoder layer $\ell = 20$ by averaging over the token dimension. We compute the steering vector as the mean difference:

$$\mathbf{v} = \frac{1}{N} \sum_{i=1}^{N} \left(h_f^{(i)} - h_r^{(i)} \right), \quad \hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|},$$

producing a unit-normalised steering vector $\hat{\mathbf{v}} \in \mathbb{R}^d$. During inference, we apply a forward hook on layer ℓ that modifies the forward pass activations at each token position:

$$h_\ell^{\text{out}} \leftarrow h_\ell^{\text{out}} + \alpha \cdot \hat{\mathbf{v}},$$

where $\alpha < 0$ is the steering strength. Setting α to a negative value subtracts the memorisation direction, which suppresses information recall without modifying the model weights. In our experiments, we use $N = 64$ contrast pairs and $\alpha = -5.0$, selected by sweeping over $\alpha \in -2.0, -5.0, -8.0, -12.0, -20.0$ which was performing the best. The steering vector comes from the baseline model fine-tuned to memorize the dataset, before unlearning, and it remains fixed throughout training and evaluation.

3.3. Adversarial Red-Teaming

To test the robustness of NPO-LoRA against adversarial attacks, we set up an adversarial red-teaming procedure. We used *gpt-4o-mini* model to rewrite the queries from the *Forget* split into seven different adversarial prompts:

- **Paraphrase** - changing the wording of the original query while keeping its meaning.
- **Prefix Injection** - prepends bypass instructions to the original query.
- **Roleplay** - presenting the query as a roleplay scenario.
- **Translation** - translating the original query into another language.
- **Code Context** - embedding the original query within a code snippet.
- **Neighbour** - focusing on nearby documents without referencing the original query.
- **Logical Chain** - using multi-step reasoning that leads back to the original query.

3.4. Evaluation Metrics

Our evaluation framework relies on the competition metrics, combining three main dimensions through an arithmetic mean to produce a final submission score. First, we calculate task-specific rates across the *Forget* and *Retain* splits. For sentence completion tasks, a ROUGE-L (Lin, 2004) regurgitation score is used, while for question-answering tasks, an Exact Match (EM) score is calculated. The metrics for the forget split are inverted ($1 -$ value). In total, 12 scores across the three tasks are then combined via the harmonic mean into a single score that represents the overall performance of the unlearning approach. Second, a Membership Inference Attack (MIA) (Shokri et al., 2017) score is calculated over member and non-member documents. This method utilises adversarial attacks to try and determine if a given document was a part of the model's training data, thus assessing its privacy protection. The MIA score is calculated using the following expression: $\text{MIA} = 1 - 2 * |\text{AUC}_{\text{MIA}} - 0.5|$, where AUC_{MIA} is the area under the ROC curve for the attack. Third, general capabilities are evaluated using the MMLU benchmark (Hendrycks et al., 2020), which consists of 57 multiple-choice tasks, representing the overall performance of the model on a wide range of general knowledge and reasoning tasks.

4. Results

We assess the performances of the **Baseline** model, model unlearned using the NPO-LoRA approach across various hyperparameters, and unlearned model with Activation Steering applied. Table 2 displays the effectiveness of unlearning using previously defined competition metrics for all models and parameter combinations, and table 3 displays the ASR summary for the **Baseline** and the chosen NPO-LoRA model across different attack types.

The details of the evaluation using competition metrics are displayed in Table 2 in a split format with three column groups: hyperparameters, base metrics, and aggregate

Table 2: Hyperparameter ablation study, each run averaged on 5 seeds

Run	Hyperparameters						Base Metrics				Aggregate Scores		
	r	α_{LoRA}	β_{NPO}	α_{NPO}	LR	Ep.	F-Reg ↓	F-Know ↓	R-Reg ↑	R-Know ↑	MIA ↑	MMLU ↑	Final ↑
Baseline	-	-	-	-	-	-	1.000	1.000	1.000	1.000	0.0	0.505	0.168
Low / Mid Capacity Adapters ($r = 8, 16$)													
1	8	16	0.05	1.00	1e-5	2	0.953	0.960	0.960	0.969	0.0	0.507	0.169
2	16	32	0.10	1.50	3e-5	1	0.933	0.809	0.948	0.872	0.0	0.503	0.223
3	16	32	0.15	1.20	5e-5	2	0.925	0.819	0.982	0.962	0.0	0.510	0.230
4	16	32	1.00	1.00	5e-5	2	0.983	0.931	0.992	0.991	0.0	0.505	0.185
High Capacity Adapters ($r = 32$)													
5	32	64	0.08	0.50	3e-5	2	0.852	0.645	0.962	0.933	0.0	0.508	0.270
6	**32**	**64**	**0.10**	**0.10**	**1e-4**	**3**	**0.720**	**0.553**	0.885	0.848	0.0	0.504	**0.318**
7	**32**	**64**	**0.10**	**0.20**	**1e-4**	**3**	0.779	0.583	**0.968**	**0.929**	0.0	0.504	0.301
8	32	64	0.30	1.00	1e-4	4	0.913	0.797	0.989	0.986	0.0	0.502	0.232
Activation Steering	32	64	0.50	1.00	1e-4	6	**0.072***	**0.004***	**0.075***	**0.010***	**0.873***	**0.499***	**0.457***

Table 3: Attack Success Rate (ASR) summary across models

Attack	ASR ↓	
	Baseline	NPO-LoRA (Run 6)
Prefix Injection	85.4%	78.2%
Roleplay	78.5%	68.1%
Paraphrase	73.7%	61.2%
Translation	68.0%	60.0%
Logical Chain	59.2%	48.9%
Neighbour	45.0%	38.7%
Code Context	1.1%	1.1%

scores. The *Hyperparameters* column shows the LoRA rank (r), LoRA scaling factor (α_{LoRA}), NPO preference strength (β_{NPO}), NPO knowledge retention weight (α_{NPO}), learning rate (LR), and the number of training epochs (Ep.). The *Base Metrics* column reports the performance on the *Forget* and *Retain* splits, where "Reg" scores refer to sentence completion tasks and "Know" scores refer to question answering tasks. The *Aggregate Scores* section presents the MIA score, MMLU benchmark score, and the final aggregate score. The arrows (↓, ↑) next to the metrics indicate the desired direction of improvement.

The **Baseline** model represents the original model without any unlearning applied. This model achieves a maximum score of 1.000 across all base metrics and a MIA score of 0.0, resulting in an (lowest) overall aggregate score of 0.168, which serves as a reference point for evaluating the effectiveness of other unlearned models.

Configurations with low to mid-capacity LoRA adapters ($r = 8, 16$) showed limited performance, with the best configuration (Run 3) achieving an aggregate score of 0.230. Increasing the LoRA adapter capacity to $r = 32$ (Runs 5-8) led to significant improvements in the aggregate score, with the best configuration (Run 6) reaching an aggregate score of 0.318. This configuration also had the lowest F-Reg (0.720) and F-Know (0.553) scores. Run 7, which had a slightly higher NPO knowledge retention weight, achieved a marginally lower aggregate score of 0.301, while maintaining higher R-Reg (0.968) and R-Know (0.929) scores, though with slightly higher F-Reg (0.779) and F-Know (0.583) scores.

Applying Activation Steering to the model unlearned using NPO-LoRA produced the best final score (0.457) across all other models, with MMLU score being slightly lower than in the rest of the experiments. Furthermore, this technique significantly improved the base metrics on the *Forget* set, making them near perfect. However, the base metrics on the *Retain* set have dropped, implying by-far the worst performance on that set.

The MIA score remained constant at its minimum (0.0) across all configurations evaluated, except for the model with Activation Steering applied, where it has the value of 0.873.

Model unlearning using NPO-LoRA approach with hyperparameters listed in run 6 of the table 2 improved the ASR across all attack types, except for the *Code Context* attacks, where it had no effect.

5. Discussion

When analyzing the results laid out in 4., we observe an inverse relationship between

6. Conclusion

Conclusion is the last enumerated section of the paper. It should not exceed half of a column and is typically split into 2–3 paragraphs. No new information should be presented in the conclusion; this section only summarizes and concludes the paper.

Acknowledgements

We would like to thank the University Computing Centre (SRCE) in Zagreb for providing the computational resources on their Advanced Computing platform used in this work.

References

João Vitor Boer Abitante, Joana Meneguzzo Pasquali, Luan Fonseca Garcia, Ewerton de Oliveira, Thomas da Silva Paula, Rodrigo C Barros, and Lucas S

Kupssinskü. 2026. Quantization-robust llm unlearning via low-rank adaptation. *arXiv preprint arXiv:2602.13151*.

Borisiuk Anna, Andrey Savchenko, Alexander Panchenko, and Elena Tutubalina. 2026. Anatomy of unlearning: The dual impact of fact salience and model fine-tuning. *arXiv preprint arXiv:2602.19612*.

Yinzhi Cao and Junfeng Yang. 2015. Towards making systems forget with machine unlearning. In *2015 IEEE symposium on security and privacy*, pages 463–480. IEEE.

Dirk Groeneveld, Iz Beltagy, Evan Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, et al. 2024. Olmo: Accelerating the science of language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15789–15809.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.

Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81.

Aengus Lynch, Phillip Guo, Aidan Ewart, Stephen Casper, and Dylan Hadfield-Menell. 2024. Eight methods to evaluate robust unlearning in llms. *arXiv preprint arXiv:2402.16835*.

Pratyush Maini, Zhili Feng, Avi Schwarzschild, Zachary C Lipton, and J Zico Kolter. 2024. Tofu: A task of fictitious unlearning for llms. *arXiv preprint arXiv:2401.06121*.

Anil Ramakrishna, Yixin Wan, Xiaomeng Jin, Kai-Wei Chang, Zhiqi Bu, Bhanukiran Vinzamuri, Volkan Cevher, Mingyi Hong, and Rahul Gupta. 2025. Lume: Llm unlearning with multitask evaluations. *arXiv preprint arXiv:2502.15097*.

Weijia Shi, Jaechan Lee, Yangsibo Huang, Sadhika Malladi, Jieyu Zhao, Ari Holtzman, Daogao Liu, Luke Zettlemoyer, Noah Smith, and Chiyuan Zhang. 2025. Muse: Machine unlearning six-way evaluation for language models. In *International Conference on Learning Representations*, volume 2025, pages 27797–27818.

Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*, pages 3–18. IEEE.

Alessandro Stolfo, Vidhisha Balachandran, Safoora Yousefi, Eric Horvitz, and Besmira Nushi. 2024. Improving instruction-following in language models through activation steering. *arXiv preprint arXiv:2410.12877*.

Vinícius Conte Turani, Otávio Parraga, João Vitor Boer Abitante, Kristen K Arguello, Joana Pasquali, Ramiro N Barros, Flavio du Pin Calmon, Christian Mattjie, Rodrigo C Barros, and Lucas S Kupssinskü. 2026. Inference-time machine unlearning via gated activation redirection. *arXiv preprint arXiv:2605.12765*.

Ruiqi Zhang, Licong Lin, Yu Bai, and Song Mei. 2024. Negative preference optimization: From catastrophic collapse to effective unlearning. *arXiv preprint arXiv:2404.05868*.